

Excel Grid View Task

Submission Expectations and Guidelines

TypeScript, HTML, CSS, Canvas, OOP, SOLID and Command Pattern

Purpose

This document clarifies what is expected at the time of submission for the Excel Grid View task. It is not a solution guide. It does not prescribe the exact implementation or provide code answers. Trainees should use this as a submission checklist, design guideline and reference list while completing the task.

1. Task Scope

Build an Excel-like grid using TypeScript, HTML, CSS and HTML Canvas. The solution should focus on clean design, maintainability and performance, not only on visible output.

Area	Expected at submission
Grid rendering	Excel-like grid rendered using HTML Canvas, including rows, columns, headers, grid lines and selected areas.
Scale	Support 100,000 rows and 500 columns using a viewport/virtual rendering approach.
Data loading	Load JSON records in the provided structure and generate/load at least 50,000 records.
Editing	Allow editing of a cell and redraw the affected area correctly.
Resizing	Support column resizing and row resizing.
Selection	Support row selection, column selection and cell-range selection.
Summary	Compute count, min, max, sum and average for numeric values in the selected range.
Undo/Redo	Typing/editing, column resizing and row resizing should support undo and redo using command pattern.
Look and feel	Match Excel-like visual behaviour as much as practical, including line thickness, headers, selected cells and resize indicators.

2. Important Constraints

- Do not create DOM elements for every cell. The grid must use Canvas for rendering.
- Do not put all logic inside one large class or one large file.
- Do not use any/untyped code unnecessarily in TypeScript.
- Do not hardcode all values as magic numbers. Use constants/configuration where suitable.
- Do not submit only a working screen. The design explanation, test cases and README are mandatory.
- Do not copy code from references without understanding and adapting it to your design.

3. Design Expectations

A short design note must be included in the submission. It can be part of the README or submitted as a separate document.

Design item	What should be explained
Class diagram / component diagram	Show the major classes and how they interact. A simple diagram is enough.
Class responsibilities	Explain the responsibility of each major class.
Data storage approach	Explain how rows, columns, cells and sparse/changed values are stored.
Virtual rendering approach	Explain how only visible rows and columns are calculated and drawn.
Command pattern approach	Explain how execute, undo and redo are handled.
Selection model	Explain how active cell, selected row, selected column and selected range are represented.
Summary calculation	Explain how count, min, max, sum and average are calculated without unnecessary full-grid processing.
Known limitations	Clearly list what is not fully implemented or needs improvement.

4. Expected Class Structure

The exact class names can vary, but the responsibilities should be clearly separated. At minimum, the solution should contain equivalents of the following components:

Component / Class	Responsibility
Grid	Main coordinator that wires data, rendering, selection, editing and commands.
GridRenderer	Draws canvas grid, headers, selected cells, active cell and resize indicators.
GridDataStore	Stores and retrieves row, column and cell data efficiently.
RowModel / ColumnModel	Represents row and column metadata such as size and index.
CellModel	Represents cell value and display-related information if needed.
ViewportManager	Calculates visible rows/columns based on scroll position and viewport size.
SelectionManager	Handles row, column, cell and range selection state.
EditManager	Handles editing workflow and input overlay coordination.
SummaryCalculator	Calculates count, min, max, sum and average for selected numeric cells.
CommandManager	Maintains undo and redo stacks.
ICommand	Interface/contract for executable and undoable actions.
EditCellCommand	Represents a cell edit action.
ResizeColumnCommand	Represents a column resize action.
ResizeRowCommand	Represents a row resize action.
JsonDataLoader	Generates and loads JSON data into the grid model.

5. OOP and SOLID Expectations

Principle	What mentors will look for
-----------	----------------------------

Single Responsibility	Each class should have a clear purpose. Rendering, selection, editing, commands and data access should not be mixed.
Open/Closed	It should be possible to add a new command, selection mode or renderer rule without rewriting large existing classes.
Liskov Substitution	If interfaces or base classes are used, derived implementations should work without surprising behaviour.
Interface Segregation	Avoid very large interfaces. Keep interfaces focused on the behaviour required.
Dependency Inversion	Higher-level classes should depend on abstractions where useful, especially for command execution, rendering contracts and data access.

6. Command Pattern and Undo/Redo Expectations

Undo and redo must be implemented using command pattern. The goal is to demonstrate design understanding, not just to manually reverse values inside event handlers.

Action	Submission expectation
Cell edit	Edit value, undo to previous value, redo to new value.
Column resize	Resize column, undo to previous width, redo to new width.
Row resize	Resize row, undo to previous height, redo to new height.
Multiple actions	Undo and redo should work in the correct order for multiple actions.
Command contract	There should be a common command interface/contract with execute and undo behaviour.

7. Performance and Large Data Expectations

The grid must be designed for scale. Rendering all 100,000 x 500 cells directly is not acceptable.

Requirement	Expected evidence
Virtual rendering	Only visible rows and columns should be drawn on canvas.
Large dataset	Show 100,000 rows and 500 columns can be represented without freezing the page.
JSON loading	Generate/load at least 50,000 records using the given JSON structure.
Scrolling	Scrolling should redraw the visible viewport, not the full grid.
Selection summary	Summary calculations should be reasonable for selected ranges and should not unnecessarily scan the full dataset.
Performance note	README should include load/render observations and any known performance limitations.

8. Functional Expectations at Submission

Feature	Minimum expected behaviour
Canvas grid	Grid lines, row headers, column headers and visible cells are drawn clearly.

Column resize	User can resize a column and the grid redraws correctly.
Row resize	User can resize a row and the grid redraws correctly.
Cell editing	User can edit a cell. Edited value persists in the grid data model.
Row selection	Click/select a row and highlight it.
Column selection	Click/select a column and highlight it.
Range selection	Select a range of cells and show visual selection.
Summary calculation	For numeric selected cells, display count, min, max, sum and average. Non-numeric values should not break calculations.
Undo/redo	Undo/redo should work for edit and resize actions.
Keyboard basics	Arrow navigation, Enter for edit, Escape to cancel edit, Ctrl+Z and Ctrl+Y for undo/redo.

9. Accessibility and Keyboard Awareness

Because Canvas is a bitmap-based drawing surface, trainees should not assume it is automatically accessible. Full accessibility is not expected in this task, but basic awareness and keyboard support are expected.

- Document the accessibility limitation of Canvas in the README.
- Support keyboard navigation for active cell movement.
- Use an HTML input overlay for cell editing rather than trying to type directly into canvas pixels.
- Show summary values as real HTML text outside the canvas.
- Make focus visible where practical.
- Avoid using color alone to communicate selection state or status without another visible cue.

10. Testing Expectations

At least 20 test scenarios must be submitted. These can be manual test cases, but they must be specific and executable.

Test area	Examples to cover
Editing	Edit empty cell, edit numeric cell with text, edit and undo, edit and redo.
Resizing	Resize column/row to normal size, very small size and undo/redo.
Selection	Single cell, row, column, visible range, range after scrolling.
Summary	Numeric-only range, mixed numeric/text range, empty selection, large selected range.
Data loading	Valid JSON, missing fields, invalid values, 50,000 generated records.
Performance	Scroll near last row, scroll near last column, load time observation.
Keyboard	Arrow keys, Enter, Escape, Ctrl+Z, Ctrl+Y.
Error handling	Invalid data or unsupported action should not crash the app.

11. Submission Package

Item	Required details
------	------------------

Source code	TypeScript, HTML and CSS source code with clean folder structure.
Generated data	JSON generation/loading logic for at least 50,000 records.
README	Setup steps, features, design explanation, SOLID usage, command pattern usage, performance notes and known limitations.
Design note/diagram	Class diagram or simple component diagram with responsibilities.
Test cases	Minimum 20 scenarios with expected result and actual result.
Screenshots/video	Evidence for grid rendering, editing, resize, selection, summary and undo/redo.
Git commits	Regular commits showing progress. Avoid one large final commit.

12. README Must Include

- Project name and objective
- How to install and run
- Features implemented
- Folder and class structure
- How OOP concepts are applied
- How SOLID principles are applied
- How command pattern is applied
- How virtual rendering works
- How data is generated and loaded
- How undo/redo works
- Test cases covered
- Performance observations
- Accessibility considerations
- Known limitations and next improvements

13. Evaluation Criteria

Area	Weightage
Canvas grid rendering and Excel-like UI	15%
Virtual rendering and performance	20%
OOP and SOLID design	20%
Command pattern and undo/redo	15%
Selection and summary calculation	10%
TypeScript quality and code structure	10%
Testing and edge-case handling	5%
README and explanation	5%

14. Mentor Review Focus

- Can the trainee explain why each major class exists?
- Is virtual rendering actually implemented, or is the code attempting to draw everything?
- Is undo/redo implemented using command pattern?

- Are TypeScript types used properly, without unnecessary any?
- Does selection remain correct after scrolling?
- Do calculations ignore non-numeric values safely?
- Does the code handle invalid data and edge cases without crashing?
- Can a new command or selection mode be added without changing many existing files?
- Is the README clear enough for another developer to run and understand the project?

15. Final Submission Checklist

- ☐ Canvas grid renders correctly
- ☐ Supports 100,000 rows and 500 columns through virtual rendering
- ☐ Loads/generated 50,000 JSON records
- ☐ Column resizing works
- ☐ Row resizing works
- ☐ Cell editing works
- ☐ Row selection works
- ☐ Column selection works
- ☐ Cell-range selection works
- ☐ Count, min, max, sum and average work for numeric selected cells
- ☐ Undo/redo works for edit, column resize and row resize
- ☐ Keyboard basics work
- ☐ Minimum expected class separation is present
- ☐ README and design note are complete
- ☐ 20 test scenarios are submitted
- ☐ Performance observations are documented
- ☐ Known limitations are mentioned honestly

16. Official References and Learning Material

Use these references for concepts and API behaviour. They are references only; the final design and implementation should be your own.

- [Canvas API - MDN Web Docs](#)
- [Basic usage of Canvas - MDN Web Docs](#)
- [HTML canvas element and accessibility notes - MDN Web Docs](#)
- [Window.requestAnimationFrame - MDN Web Docs](#)
- [Pointer Events - MDN Web Docs](#)
- [KeyboardEvent - MDN Web Docs](#)
- [Performance.mark - MDN Web Docs](#)
- [Performance.measure - MDN Web Docs](#)
- [TypeScript Handbook - Documentation](#)
- [TypeScript Interfaces - Handbook](#)
- [TypeScript Generics - Handbook](#)
- [TSConfig strict option - TypeScript Docs](#)
- [WAI-ARIA Grid Pattern - W3C](#)